Scenario 3 — CPU Pressure

Routing Behaviour Under Elevated Backend CPU Utilization

Scenario	`cpu_pressure`
Data source	`bench/results/cpu_pressure_summary.json` and 20 per-run JSON files
Measurement	5 s warm-up and 60 s measured interval per run
Design	4 routing strategies × 5 repetitions = 20 runs
Data verified	20 October 2026

Abstract

This report evaluates whether the Adaptive routing strategy detects elevated CPU utilization on one backend and shifts traffic away from it. Three backends were configured with an identical 20 ms service latency and weights of 3:1:2. At 30 s into each 60 s measurement interval, one CPU worker was started on `backend-3`. All four strategies were repeated five times at 300 requests per second with 32 workers. The mean latency remained between 20.94 ms and 21.09 ms, with zero failed requests. Backend CPU measurements show a clear but intermittent CPU signal on `backend-3`: samples reached approximately 5.0%, while `backend-1` and `backend-2` never exceeded 0.07%. Despite this signal, Adaptive did not move traffic away from the pressured backend: its `backend-3` share rose from 33.48% before the phase to 34.32% during the phase. The experiment therefore produced a valid negative result for end-to-end CPU-aware routing: CPU pressure was observable at the metric level, but it neither changed Adaptive's routing distribution nor improved latency.

1 Scenario Objective

This scenario isolates CPU utilization as a routing signal. The three backends have identical configured service latency (20 ms) and therefore no intrinsic latency advantage. In the second half of each run, a single CPU-bound worker is started on `backend-3`. The central question is:

> *Can the Adaptive strategy detect increased backend CPU utilization and shift traffic away from the CPU-pressured backend?*

The question matters because CPU utilization is an explicit input to the Adaptive strategy's V1 score, alongside active connections and response time. Unlike Scenario 2, which directly changes request service latency, Scenario 3 changes a resource signal that is sampled independently of the request path. It therefore tests the translation chain from *resource pressure* to *measured CPU metric* to *Adaptive score* to *routing decision*. A change in that chain is not guaranteed merely because a CPU burner is active.

Two outcomes must be distinguished throughout the report. *Metric-level adaptation* means that the measured CPU signal changed and that Adaptive's selection distribution changed in response. *End-to-end improvement* means that latency, errors, or tail behaviour improved. This experiment tests both, and it is possible for the first to occur without the second — or for neither to occur.

2 Experimental Configuration

The configuration below is taken directly from `bench/scenarios/cpu_pressure.json`. No values in this section are inferred from terminal output.

Table 1: Actual Scenario 3 configuration.

Parameter	Value
Scenario name	`cpu_pressure`
Purpose (file description)	Artificially increase CPU pressure on `backend-3` mid-run
Backends	3: `backend-1` (port 18081), `backend-2` (18082), `backend-3` (18083)
Backend weights	3 : 1 : 2
Baseline backend latency	20 ms on every backend
CPU-pressure phase	At 30 s: `set_cpu` on `backend-3` with 1 CPU worker
Request rate	300 requests/s (open loop)
Concurrency / workers	32
Warm-up	5 s (excluded from measurement)
Measurement duration	60 s
Repetitions	5 per strategy
Strategies	Round Robin, Least Connections, Weighted Round Robin, Adaptive
Total runs	20 (4 strategies × 5 repetitions)
Expected requests per run	18,000 (300 req/s × 60 s)
CPU pinning	None defined in the scenario; the standard runner applies no pinning unless `--cpu-sets` is supplied

The phase time is relative to the start of the measured interval, after the 5 s warm-up has been discarded. The CPU worker therefore runs for approximately the final 30 s of each 60 s measurement. It remains active until the backend process is stopped at the end of the run; there is no second phase that removes the load. The scenario defines no latency change, no backend removal, and no change to the request rate.

The benchmark host exposed 20 logical CPUs. Because the backend CPU metric is normalized by the number of logical CPUs, one fully busy core is represented as approximately $1/20 = 5\%$ utilization, not 100%. This normalization is important when interpreting both the CPU tables and Adaptive's score.

3 Benchmark Methodology

3.1 Workload generation

Each run starts three backend processes and one load-balancer process, waits for their health endpoints, and then executes the workload. After the warm-up, the runner emits requests from a ticker at 300 requests per second. A fixed pool of 32 workers consumes those requests, giving an open-loop arrival process with a bounded worker pool. Each request is a `GET /` through the load balancer; the load balancer selects a backend according to the strategy under test. The response header `X-Backend-ID` identifies the selected backend.

Results are accumulated over 60 one-second windows. The warm-up requests are discarded. Latency is measured around the complete HTTP request, including connection reuse, proxying, backend selection, and the configured 20 ms backend delay.

3.2 CPU pressure generation

At 30 s after measurement start, the orchestrator sends `POST /bench/cpu?workers=1` to `backend-3`. The backend starts one CPU-bound goroutine that repeatedly performs integer mixing and periodically updates an atomic sink. The burner is independent of the HTTP handler: it does not add queueing to request processing, does not change the configured latency, and does not reject requests. This design intentionally isolates CPU *utilization* from CPU-induced *service-time* effects.

3.3 CPU collection

The backend samples its own CPU utilization with a 500 ms measurement window every 2 s and exposes the latest value at `/metrics/cpu`. The load balancer's CPU collector polls each backend every 5 s and stores the value on the corresponding server object. The Adaptive strategy treats a CPU sample as fresh for 15 s. Independently, the workload runner polls the same endpoint every second and records the latest value in each one-second result window.

The utilization value is computed from `/proc/self/schedstat` and divided by `runtime.NumCPU()`. On Linux, this proc file is thread-scoped: it reports scheduler time for the task performing the read, not an aggregate over all threads in the process. A controlled check of the same sampling path showed process-wide CPU at a stable 5.01% while the schedstat-derived value oscillated between 0% and 5%. This detail explains the intermittency reported in Section 6 and is treated as a measurement-path limitation, not as evidence that the burner itself was intermittent.

3.4 Routing comparison and aggregation

All four strategies are executed against the same configuration and workload. Each strategy is repeated five times, with the strategy order rotated between repetitions. For each strategy the report uses:

- the five *per-run mean latencies*;
- the *median* of those five means, used as the central estimate;
- the *interquartile range* (IQR, $Q_3 - Q_1$) of the five means;
- the *pooled mean*, weighted by the number of requests in each run;
- the median of the five per-run P50, P95, and P99 values;
- the aggregate error count and error rate;
- backend request counts, overall and split into the pre-phase (0–29 s) and post-phase (30–59 s) intervals.

All numerical values in this report were computed from the individual `cpu_pressure_*_run*.json` files. The summary file was checked against those files and found to contain the same 20 result objects.

4 Overall Latency Results

4.1 Summary by strategy

Table 2 reports the complete latency comparison. The terminal output suggested values of approximately 21.0 ms, 21.1 ms, 20.95 ms, and 21.06 ms; the exact medians derived from the JSON files are 20.981 ms, 21.093 ms, 20.937 ms, and 21.052 ms respectively.

Table 2: Latency summary by strategy. Median and IQR are computed over the five per-run mean latencies. P50/P95/P99 are medians of the per-run percentile values. The pooled mean weights each run by its request count. Request totals are reported in the text and in Section 12.

Strategy	Median (ms)	IQR (ms)	Pooled mean (ms)	P50 (ms)	P95 (ms)	P99 (ms)
Round Robin	**20.981**	0.015	20.983	21.0	21.7	21.9
Least Connections	**21.093**	0.019	21.093	21.2	21.6	21.8
Weighted Round Robin	**20.937**	0.033	20.948	20.9	21.6	21.9
Adaptive	**21.052**	0.018	21.064	21.1	21.6	21.9

All 359,987 requests completed successfully: 89,997 for Round Robin, 89,996 for Least Connections, 89,998 for Weighted Round Robin, and 89,996 for Adaptive. Every strategy had an error count of zero, i.e. an error rate of 0.000%. The per-strategy coefficient of variation of the five run means was 0.084% for Round Robin, 0.055% for Least Connections, 0.098% for Weighted Round Robin, and 0.114% for Adaptive.

4.2 Per-run measurements

Table 3: Per-run mean latency by strategy (ms).

Strategy	Run 1	Run 2	Run 3	Run 4	Run 5
Round Robin	21.009	20.962	20.990	20.981	20.974
Least Connections	21.104	21.105	21.079	21.093	21.086
Weighted Round Robin	20.970	20.937	20.969	20.936	20.926
Adaptive	21.105	21.049	21.047	21.067	21.052

The run-to-run spread is small for every strategy. The largest within-strategy range is 0.058 ms (Adaptive), which is smaller than the 0.1 ms resolution of the reported percentile values. Weighted Round Robin has the lowest median by a small margin; Adaptive is 0.114 ms slower than Weighted Round Robin and 0.071 ms slower than Round Robin. These differences are examined in Section 9.

4.3 Latency before and during CPU pressure

To test the effect of the CPU phase directly, each run was split at the phase boundary. Table 4 shows request-weighted mean latency and the medians of the per-window P95/P99 values for the first and second 30 s. Windows 0–29 are pre-phase; windows 30–59 are post-phase. Because CPU collection and routing updates are periodic, the first few post-phase windows may still contain pre-phase CPU values, making this a conservative test.

Table 4: Latency before and during the CPU-pressure phase. Means are request-weighted across windows; P95/P99 are medians of window percentiles.

Strategy	Pre mean	Post mean	Change	Pre P95	Post P95	Pre P99 / Post P99
Round Robin	21.105	20.862	−0.243	21.75	21.5	21.9 / 21.6
Least Connections	21.149	21.038	−0.110	21.70	21.5	21.8 / 21.6
Weighted Round Robin	21.091	20.805	−0.285	21.70	21.5	21.9 / 21.6
Adaptive	21.158	20.970	−0.188	21.70	21.5	21.9 / 21.6

The post-phase means are 0.11–0.29 ms *lower* than the pre-phase means, not higher. The same pattern appears in the window-level P95 and P99 values. This is the opposite of the direction expected if CPU contention were delaying request service. The effect is small and is consistent with ordinary drift in the measurement environment; it provides no evidence of CPU-induced latency degradation.

Table 5: Backend request distribution over the complete run. Percentages are within each strategy.

	`backend-1`		`backend-2`		`backend-3`	
Strategy	Count	%	Count	%	Count	%
Round Robin	30,000	33.33	30,000	33.33	29,997	33.33
Least Connections	38,400	42.67	25,844	28.72	25,752	28.61
Weighted Round Robin	45,000	50.00	15,000	16.67	29,998	33.33
Adaptive	30,016	33.35	29,470	32.75	30,510	33.90

5 Backend Request Distribution

5.1 Overall distribution

Table 5 gives the aggregate request counts and percentages for the complete 60 s run. These values include both the pre-phase and post-phase intervals.

Round Robin is essentially uniform, as expected from its rotation. Weighted Round Robin reproduces the configured 3:1:2 weights almost exactly. Least Connections strongly favours `backend-1`; this is a structural property of its connection accounting under this workload, not a CPU response, because the preference is already present before the CPU phase. Adaptive is close to uniform, with a small excess on `backend-3` over the full run.

5.2 Distribution before and during CPU pressure

The critical test is whether the distribution changes after CPU pressure is applied. Table 6 reports the share of requests received by each backend in the pre-phase and post-phase intervals.

Table 6: Backend request share before and during the CPU-pressure phase. Values are percentages of requests in the corresponding 30 s interval.

	Pre-phase (0–29 s)			Post-phase (30–59 s)		
Strategy	B1	B2	B3	B1	B2	B3
Round Robin	33.34	33.34	33.33	33.33	33.33	33.34
Least Connections	42.60	28.77	28.63	42.74	28.66	28.60
Weighted Round Robin	50.01	16.67	33.33	50.00	16.67	33.34
Adaptive	33.26	33.26	33.48	33.44	32.24	34.32

Round Robin and Weighted Round Robin are invariant across the phase boundary, which confirms that the CPU phase did not alter the workload or the basic routing logic. Least Connections also remains at approximately 42.7/28.7/28.6. Adaptive's post-phase `backend-3` share is 34.32%, compared with 33.48% before the phase: a rise of 0.84 percentage points. Its `backend-2` share falls by 1.02 percentage points and its `backend-1` share rises by 0.18 percentage points.

The per-run Adaptive shares show the same picture. Post-phase `backend-3` share is 33.56%, 34.93%, 33.66%, 34.22%, and 35.24% in runs 1–5; the corresponding pre-phase values are 33.27%,

33.72%, 33.66%, 33.59%, and 33.17%. Four of the five runs *increase* the pressured backend's share; one is unchanged. There is no run in which Adaptive shifts traffic away from `backend-3` in a sustained way.

This is a decisive result: the measured distribution does *not* support the claim that Adaptive avoided the CPU-pressured backend. The small change in shares is in the opposite direction and is of the same order as the run-to-run variation seen in a near-uniform allocation.

6 Backend CPU Measurements

6.1 Observed CPU levels

Table 7 summarises the CPU values recorded in the result windows. "Post mean" is the mean over all 150 post-phase windows per strategy (5 runs × 30 windows). Detection rates and detected-sample means for samples above 1% — a threshold well above every non-pressured sample observed in this experiment — are reported in the text below the table.

Table 7: Backend CPU utilization before and during the CPU-pressure phase (%). Values are percentages of total host CPU capacity. "Pre" is the `backend-3` mean during windows 0–29; other columns use windows 30–59.

	B1 post		B2 post		B3 post		B3 pre	
Strategy	Mean	Max	Mean	Max	Mean	Max	Mean	Max
Round Robin	0.005	0.031	0.009	0.037	0.551	4.996	0.018	0.051
Least Connections	0.015	0.051	0.002	0.032	0.374	4.998	0.015	0.048
Weighted Round Robin	0.012	0.068	0.006	0.026	0.802	4.991	0.025	0.055
Adaptive	0.007	0.035	0.009	0.037	0.750	4.997	0.028	0.062

`backend-3` is the only backend that shows elevated CPU. Its post-phase maximum is 4.99–5.00% in every strategy, whereas `backend-1` and `backend-2` never exceed 0.068% in the post-phase windows and never produce a sample above 0.1%. The CPU pressure is therefore distinguishable in magnitude: it reaches the equivalent of one fully busy logical CPU, and it is roughly two orders of magnitude above the background values on the other backends.

The signal is nevertheless highly intermittent. The median post-phase `backend-3` value is between 0.000% and 0.019% depending on strategy, and only 14.7–24.0% of one-second windows exceed 1%. The means of the detected samples are 2.3–3.5%, with peaks near 5%. This is not a stable elevated plateau; it is a sequence of mostly zero or near-zero values punctuated by full-core samples. When only the clear detections above 1% are considered, the mean is 2.931% for Round Robin, 2.317% for Least Connections, 3.128% for Weighted Round Robin, and 3.453% for Adaptive; the corresponding detection rates are 17.3%, 14.7%, 24.0%, and 20.0%.

6.2 Run-to-run behaviour and time series

Table 8 shows the post-phase `backend-3` mean for each run. The variation across runs is large relative to the differences between strategies. Since the burner configuration is identical in every run, these cross-strategy differences should not be read as a routing effect; they are sampling variability in how often the thread-scoped CPU reading captured the burner.

Table 8: Post-phase mean CPU utilization of `backend-3` by run (%). The phase is active for windows 30–59.

Strategy	Run 1	Run 2	Run 3	Run 4	Run 5
Round Robin	0.334	1.078	0.376	0.554	0.410
Least Connections	0.673	0.029	0.803	0.004	0.361
Weighted Round Robin	1.061	1.049	0.223	0.824	0.850
Adaptive	0.810	1.136	0.610	0.404	0.789

Table 9 aggregates the Adaptive runs into five-second bins. Before 30 s, the mean stays between 0.015% and 0.041%. After 30 s, all bins contain elevated values, but no stable plateau emerges: the bin means range from 0.212% to 1.292%, and the bin maxima range from 3.386% to 4.997%.

Table 9: Adaptive runs: mean and maximum `backend-3` CPU utilization in five-second bins (%). Each bin pools five runs.

Interval (s)	Mean	Maximum
0–4	0.035	0.053
5–9	0.021	0.048
10–14	0.015	0.056
15–19	0.025	0.062
20–24	0.041	0.061
25–29	0.029	0.053
30–34	0.493	4.997
35–39	1.292	4.841
40–44	0.937	4.807
45–49	0.569	4.807
50–54	0.212	3.386
55–59	0.996	4.317

The first post-phase window containing a `backend-3` sample above 0.1% ranged from window 31 to window 54 across all runs and strategies; most runs first showed a clear sample between windows 32 and 43. The lag reflects the backend's 2 s sampling interval, the load balancer's 5 s collection interval, and the intermittent thread-scoped reading. It is not evidence that the burner started late: the phase request is issued at 30 s by the orchestrator.

6.3 Measurement-path caveat

The backend's sampler reads `/proc/self/schedstat` and normalizes the delta by the number of logical CPUs. On Linux, that file describes the scheduler time of the task executing the read, not the sum across all threads of a multi-threaded process. The CPU burner runs in a separate goroutine, which the Go scheduler can place on a different OS thread. Consequently, a nominally continuous one-core burn can appear as 0% in many 500 ms samples and as approximately 5%

in others. A direct check of the sampler path confirmed this: process-wide utilization measured with `getrusage(RUSAGE_SELF)` stayed at 5.01%, while the schedstat-derived reading alternated between 0% and 5%.

Two conclusions follow. First, the CPU pressure was genuinely present during the post-phase interval; the zeros are a measurement artefact. Second, the signal available to Adaptive was much noisier than the physical CPU load. The load balancer also polls only every 5 s, so a 500 ms high sample can be missed entirely between collections. This is a limitation of the current V1 measurement path and a plausible contributor to the absence of a routing response.

7 Adaptive Strategy Analysis

7.1 V1 scoring model

Adaptive V1 assigns the following weights to its three normalized signals:

$$S_i = 0.45\,C_i + 0.40\,R_i + 0.15\,U_i, \tag{1}$$

where C_i is the min–max-normalized active-connection count, R_i is the min–max-normalized response time, and $U_i = \mathrm{CPU}_i/100$ is the CPU utilization expressed as a fraction. Lower scores are preferred. CPU has the lowest weight because it is a coarse, delayed, and externally sampled signal relative to per-request connection and latency observations.

The weight alone understates how small the CPU contribution is in this experiment. One busy logical CPU on a 20-CPU host produces a CPU metric of about 5%, so $U_i \approx 0.05$. The weighted CPU term is therefore $0.15 \times 0.05 = 0.0075$. For the non-pressured backends, whose observed values are below 0.07%, the term is below 0.0001. The maximum CPU-induced score separation is thus roughly 0.0075 on a 0–1 score scale, or 0.75% of the full scale.

By comparison, the connection term can vary over the full 0–1 normalized range with weight 0.45, and the response-time term has weight 0.40. Even a modest normalized difference in either signal can exceed the entire CPU effect. The CPU term is capable of breaking a near-tie (the score epsilon is 10^{-6}), but it is not capable of overriding a meaningful connection or response-time difference.

7.2 Observed routing response

The measured distribution shows that the CPU signal did not produce avoidance. Adaptive's `backend-3` share increased by 0.84 percentage points during the phase, and its overall distribution remained close to uniform (33.35%, 32.75%, 33.90%). If the CPU term had dominated, the expected signature would be a sustained reduction in `backend-3` traffic after 30 s, visible in both the pooled post-phase share and most individual runs. Neither signature is present.

The most defensible interpretation is not that Adaptive is "CPU-blind" in principle. The code does consume the CPU signal and does apply the 0.15 weight. Rather, under this workload the signal was (i) intermittently observed because of the thread-scoped sampler, (ii) small in score terms even at its 5% peak, and (iii) competing with connection and response-time terms that vary more. The experiment therefore tests the V1 design at a CPU load and measurement granularity where the CPU channel has little chance to dominate. It does not establish that Adaptive would ignore a larger, stable, correctly measured CPU difference.

8 Why Latency May Remain Similar

The latency results are nearly identical across strategies because the CPU pressure was not coupled to request service time in a way that could produce queueing delay. The following explanations are supported by the measured data and by the configured workload:

1. **CPU pressure was independent of request handling.** The burner runs in a separate goroutine and never blocks, delays, or rejects an HTTP request. The handler continues to sleep for the configured 20 ms. The request path therefore has no mechanism by which the burner can add service time, except through host-level contention.
2. **The backend retained ample CPU capacity.** The host has 20 logical CPUs and the burner occupies at most one core. At 300 req/s with a 20 ms service time, the system has roughly six requests in flight in total and about two per backend on average. This is far below the capacity at which CPU contention would create a request queue.
3. **The workload was too light to expose contention.** A mean latency of about 21 ms is essentially the configured 20 ms delay plus HTTP and proxy overhead. There is no measured latency inflation, no error increase, and no tail growth during the phase; the post-phase means are slightly lower, not higher.
4. **The CPU signal was intermittent at the routing layer.** The thread-scoped sampler produced many 0% readings, and the load balancer polls every 5 s. Adaptive therefore often saw no CPU difference at all. When it did see a difference, the 0.15 weight and the $U_i = \text{CPU}/100$ normalization limited the score effect to at most about 0.0075.
5. **The 15% CPU weight did not dominate the other signals.** Even a full-core sample contributed less than one percentage point of the score scale. Connection and response-time differences can be larger, so CPU could not reliably redirect traffic on its own.

A crucial distinction must be maintained. *CPU pressure was detected*: `backend-3` reached approximately 5% while the other backends stayed below 0.07%, and this is a real signal in the recorded data. *CPU pressure did not affect request latency*: the end-to-end means, P95, and P99 were unchanged or slightly lower during the phase. The experiment therefore demonstrates a metric-level CPU event without an end-to-end performance effect. It does not demonstrate that CPU pressure is harmless in general; it shows that this particular pressure configuration, at

this load and measurement granularity, was not translated into request delay.

9 Comparative Analysis

Table 10 reports the differences in median mean latency and pooled mean latency. Positive values mean that the first strategy is slower.

Table 10: Quantitative latency comparisons. Differences are in milliseconds; percentages are relative to the comparator.

Comparison	Δ median	Δ median (%)	Δ pooled	Δ pooled (%)
Adaptive vs. Round Robin	+0.071	+0.34	+0.081	+0.39
Adaptive vs. Least Connections	−0.042	−0.20	−0.029	−0.14
Adaptive vs. Weighted Round Robin	+0.114	+0.55	+0.116	+0.55
Least Connections vs. Round Robin	+0.113	+0.54	+0.110	+0.53
Weighted Round Robin vs. Round Robin	−0.043	−0.21	−0.035	−0.17

The largest difference involving Adaptive is 0.114 ms against Weighted Round Robin, or 0.55%. The run-to-run IQRs are 0.015–0.033 ms, and the per-run ranges are 0.027–0.058 ms. A difference of 0.114 ms is therefore comparable to, or smaller than, the normal variation within a strategy. It is not practically meaningful for this workload.

The ranking is also not consistent with a CPU-aware advantage. Adaptive is slightly slower than Round Robin and Weighted Round Robin, and only 0.042 ms faster than Least Connections. Least Connections has the highest mean because its distribution sends more traffic to `backend-1`; since all backends have the same 20 ms service time, this does not produce a latency penalty large enough to matter, and the observed difference is sub-millisecond. None of these differences supports a claim that Adaptive outperformed the simpler strategies.

10 Tail Latency

Table 11 reports the per-strategy P95 and P99 medians and the median of the per-run maximum latencies. The P95 values differ by at most 0.1 ms and the P99 values by at most 0.1 ms.

Table 11: Tail-latency summary. P95/P99 are medians of the five per-run values; maximum is the median of the five per-run maxima.

Strategy	P95 (ms)	P99 (ms)	Median maximum (ms)
Round Robin	21.7	21.9	25.05
Least Connections	21.6	21.8	27.53
Weighted Round Robin	21.6	21.9	26.45
Adaptive	21.6	21.9	25.21

The tail values are essentially the configured 20 ms service time plus a small and stable overhead.

The post-phase window-level tails are slightly lower than the pre-phase tails for every strategy (P95 21.5 vs. 21.7 ms; P99 21.6 vs. 21.9 ms in the aggregate windows). There is no evidence that CPU pressure produced a tail-latency penalty for any strategy, and no evidence that Adaptive reduced tail latency relative to the others. The isolated per-run maxima up to 31.6 ms (Weighted Round Robin) are rare events below the P99 and are not concentrated in the CPU phase.

11 Scenario 1 / 2 / 3 Comparison

The three scenarios probe different routing challenges. Table 12 summarises their designs and principal outcomes.

Table 12: Comparison of the three benchmark scenarios. Values are medians of per-run mean latency unless noted.

Scenario	Backend configuration	Purpose	Principal result
1. Baseline	3 × 20 ms; 3:1:2; 300 req/s	Neutral baseline with homogeneous latency	All strategies 21.21–21.27 ms; differences at most 0.06 ms. No strategy has a meaningful advantage.
2. Heterogeneous latency	50/100/200 ms; 3:1:2; 150 req/s	Test latency-aware routing under a strongly heterogeneous backend mix	RR 118.68 ms, LC 84.60 ms, WRR 110.39 ms, Adaptive 84.88 ms. LC and Adaptive reduce mean latency by about 28% versus RR by avoiding the 200 ms backend.
3. CPU pressure	3 × 20 ms; 3:1:2; 300 req/s; one CPU worker on B3 at 30 s	Test CPU-aware routing when service latency is unchanged	All strategies 20.94–21.09 ms. CPU pressure is visible on B3 (up to 5.0%), but Adaptive's B3 share does not fall; no end-to-end benefit.

Scenario 2 creates a genuine routing challenge because the backend service times differ by a factor of four. The routing decision has an immediate, measurable effect on request latency: sending a request to the 200 ms backend instead of the 50 ms backend adds 150 ms. Both Least Connections and Adaptive exploit this and reduce the mean substantially. Scenario 3 is different in kind. Its CPU signal is real, but the request service time remains 20 ms on every backend, and the CPU burner is not on the request's critical path. Consequently, even a correct CPU-aware routing decision would not be expected to reduce latency much; the measured distribution shows that Adaptive did not make such a decision in any case.

Scenario 3 is therefore a distinct *experimental* challenge — it tests a different signal and a different part of the Adaptive design — but it did not create a distinct *effective* routing challenge in terms of end-to-end performance. It is best interpreted as a test of metric response under CPU pressure, not as a replication of Scenario 2's latency-separation test. The negative result is consistent with the design: a 15%-weighted, intermittently sampled, one-core signal cannot be expected to dominate a system whose service time is fixed and whose connection and response-time signals vary more.

12 Benchmark Validity

12.1 Completeness and agreement

All 20 expected runs are present: four strategies with five repetitions each. The summary file contains exactly 20 result objects, and every object is identical to the corresponding per-run JSON. There are no missing runs, no missing result files, and no partial JSON files. The per-run logs contain only backend startup lines because the load balancer discards logs when not run in verbose mode; this is expected and does not indicate a failed run. The orchestrator wrote a complete JSON for every run.

12.2 Request counts

The nominal request count is 18,000 per run (300 req/s for 60 s). The observed counts are 17,998–18,000. Table 13 lists every run.

Table 13: Observed requests per run.

Strategy	Run 1	Run 2	Run 3	Run 4	Run 5
Round Robin	17,999	17,999	17,999	18,000	18,000
Least Connections	17,999	17,999	18,000	18,000	17,998
Weighted Round Robin	18,000	18,000	18,000	17,999	17,999
Adaptive	17,999	17,999	17,998	18,000	18,000

The total is 359,987 requests versus a nominal 360,000, a shortfall of 13 requests or 0.0036%. The largest per-run deviation is two requests (0.0111%). The first and last one-second windows contain 293 and 307 requests respectively rather than exactly 300, reflecting ticker alignment at the measurement boundaries; the aggregate remains consistent with a 300 req/s open-loop schedule. This deviation is negligible and cannot affect the conclusions. It is not treated as a failed run.

12.3 Errors, consistency, and anomalies

All runs have zero errors. The per-strategy coefficients of variation are 0.055–0.114%, and the IQRs are 0.015–0.033 ms, indicating stable repeated measurements. Backend distributions are also stable: Round Robin is exactly uniform, Weighted Round Robin follows 3:1:2, Least Connections is consistently about 42.7/28.7/28.6, and Adaptive remains close to uniform with only small run-to-run movement.

CPU data are present in all 60 windows of all 20 runs, for all three backends; there are no missing CPU keys. The CPU values are, however, intermittent by construction of the sampler, as discussed in Section 6. This is a measurement consistency issue, not missing data. The most visible anomalous run is Least Connections run 4, whose post-phase `backend-3` mean is only 0.004% and which never records a sample above 0.1%. Its latency, error count, and request distribution are normal, consistent with the sampler missing the burner thread.

13 Limitations

The following limitations are specific to this experiment and are relevant to the interpretation of the negative result:

- **CPU is only 15% of the score.** Even a full-core CPU signal is multiplied by 0.15 after normalization, and a 5% metric becomes only 0.0075 of the score scale.
- **The CPU signal is sampled periodically and thread-scoped.** The backend samples for 500 ms every 2 s, the load balancer polls every 5 s, and the schedstat read does not aggregate all threads. The routing layer therefore sees a noisy, delayed, and sometimes zero signal.
- **CPU utilization is process/thread-level, not request-level.** The metric does not isolate the CPU cost of individual requests, and it is not a direct measure of queueing or saturation on the request path.
- **CPU pressure is independent of service time.** The burner runs beside the handler rather than delaying it. A CPU-pressure experiment that does not create request queueing cannot be expected to change latency unless the host is genuinely saturated.
- **The workload is light and the host has spare capacity.** At 300 req/s, 20 ms service time, and 20 logical CPUs, one busy core leaves substantial headroom. The result may differ at higher load or with a smaller CPU allocation.
- **Only one pressure configuration is tested.** There is a single transition (one worker on one backend at 30 s), no ramp, no stronger multi-core pressure, and no second phase that removes pressure.
- **Only three backends and one host are used.** The experiment is a local loopback benchmark. It does not cover network effects, multiple hosts, container CPU quotas, or noisy-neighbour interference.
- **Routing decisions are inferred from aggregate counts.** The per-run logs do not contain per-request selection traces or the load balancer's sampled CPU values, so the analysis uses the recorded backend distribution and the workload's CPU snapshots.

These limitations do not invalidate the measured outcome. They define the conditions under which the outcome was obtained and explain why a negative result is plausible for the current V1 design.

14 Conclusion

This experiment did *not* demonstrate that CPU-aware Adaptive routing provides a measurable end-to-end advantage. The CPU-pressure phase did produce a real, distinguishable metric-level signal: `backend-3` reached approximately 5.0% CPU utilization, while `backend-1` and `backend-2` remained below 0.07%. However, the signal was intermittent, appearing above 1% in only 14.7–24.0% of post-phase windows, and Adaptive did not shift traffic away from the pressured backend. Its `backend-3` share was 33.48% before the phase and 34.32% during the phase, a small increase rather than a reduction. The distribution therefore contradicts the hypothesis that Adaptive avoided the CPU-pressured backend.

The latency evidence is consistent with this interpretation. Mean latency was 20.94–21.09 ms across all strategies, with zero errors; Adaptive's median (21.052 ms) was 0.34% slower than Round Robin and 0.55% slower than Weighted Round Robin, and only 0.20% faster than Least Connections. These differences are smaller than the normal run-to-run variation and are not practically meaningful. P95 and P99 differed by at most 0.1 ms, and the post-phase window tails were slightly lower than the pre-phase tails. The experiment demonstrates that CPU pressure was visible in the measurement data without being translated into request latency or a routing response.

The result is a valid negative finding for the current V1 design under this workload. The likely explanation is structural rather than a failure of the benchmark: one busy core on a 20-CPU host maps to only about 5% in the CPU metric, the CPU weight is 0.15, and the resulting score contribution (about 0.0075) is too small to override connection and response-time differences. The thread-scoped sampling and 5 s polling interval further reduce the chance that the routing layer sees a stable elevated value. A stronger test of CPU-aware routing would require a CPU signal that is both stable and material to request service — for example, sustained multi-core pressure, CPU quotas that actually throttle the handler, or a higher CPU weight in a controlled variant. Those changes are outside the scope of this experiment and were not made here.